



Yunnan University, School of Software and AI
Software Service Engineering (YN3012180059)

Final Assignment Report

Student ID	Name	Major	Score
20233120013	Tuhin Md Abu Hamza	Software	

Semester: Spring 2026

Teacher: Ahmed Zahir

Topic: [Food Delivery Management System]

Submission Date: 2025/06/22

Table Of Contents

Abstract.....	2
1. Introduction.....	2
1.1 Background.....	2
1.2 Objectives.....	2
1.3 Scope.....	2
2. System Architecture.....	3
2.1 Microservices Architecture.....	3
2.2 Communication Patterns.....	4
2.3 Data Flow.....	4
3. Implementation Details.....	4
3.1 Order Service.....	4
3.2 Restaurant Service.....	5
3.3 Notification Service.....	5
3.4 DTO Mapping with MapStruct.....	6
3.5 API Documentation.....	6
4. Cloud-Native Concepts.....	7
4.1 Service Discovery (Netflix Eureka).....	7
4.2 API Gateway (Spring Cloud Gateway).....	7
4.3 API Security (JWT).....	7
4.4 Centralized Configuration.....	7
4.5 Resilience (Resilience4j).....	7
4.6 Distributed Tracing (Zipkin).....	7
5. Message-Driven Communication.....	7
5.1 Kafka Integration.....	7
5.2 RabbitMQ Integration.....	8
6. Testing.....	8
6.1 Unit Testing.....	8
6.2 Integration Testing.....	8
6.3 Semi-Manual Testing.....	8
7. Docker Containerization.....	9
7.1 Container Architecture.....	9
7.2 Docker Compose.....	10
8. Challenges and Solutions.....	10
9. Conclusion.....	11
10. References.....	11

Abstract

This report presents the design and implementation of a cloud-native reactive microservices-based Food Delivery Management System using Spring Boot and Spring Cloud. The system consists of three microservices: Order Service, Restaurant Service, and Notification Service. It implements service discovery via Eureka, API Gateway, asynchronous communication using Kafka and RabbitMQ, and database per service pattern with PostgreSQL and MongoDB. All services are containerized with Docker. The system demonstrates reactive programming, RESTful API design, and cloud-native principles including security, resilience, and distributed tracing.

Keywords: Microservices, Reactive Programming, Spring Boot, Spring Cloud, Kafka, RabbitMQ, Docker, Cloud-Native

GitHub Repository: <https://github.com/ahtbd/food-delivery-system>

1. Introduction

1.1 Background

The Food Delivery Management System is a cloud-native reactive microservices-based application built using Spring Boot and Spring Cloud technologies. This project demonstrates the implementation of a scalable, maintainable, and loosely coupled distributed system following industry best practices [1][4].

1.2 Objectives

The primary objectives of this project are:

1. To design and implement a cloud-ready reactive microservices-based application
2. To demonstrate understanding of reactive microservices architecture
3. To implement RESTful API design with proper documentation
4. To integrate service discovery and API Gateway
5. To implement message-driven communication using Kafka and RabbitMQ
6. To containerize all services using Docker
7. To incorporate cloud-native design principles including security, resilience, and observability

1.3 Scope

The system covers the complete food delivery workflow:

1. Restaurant registration and menu management
2. Order creation and status tracking
3. Notification delivery for order updates

2. System Architecture

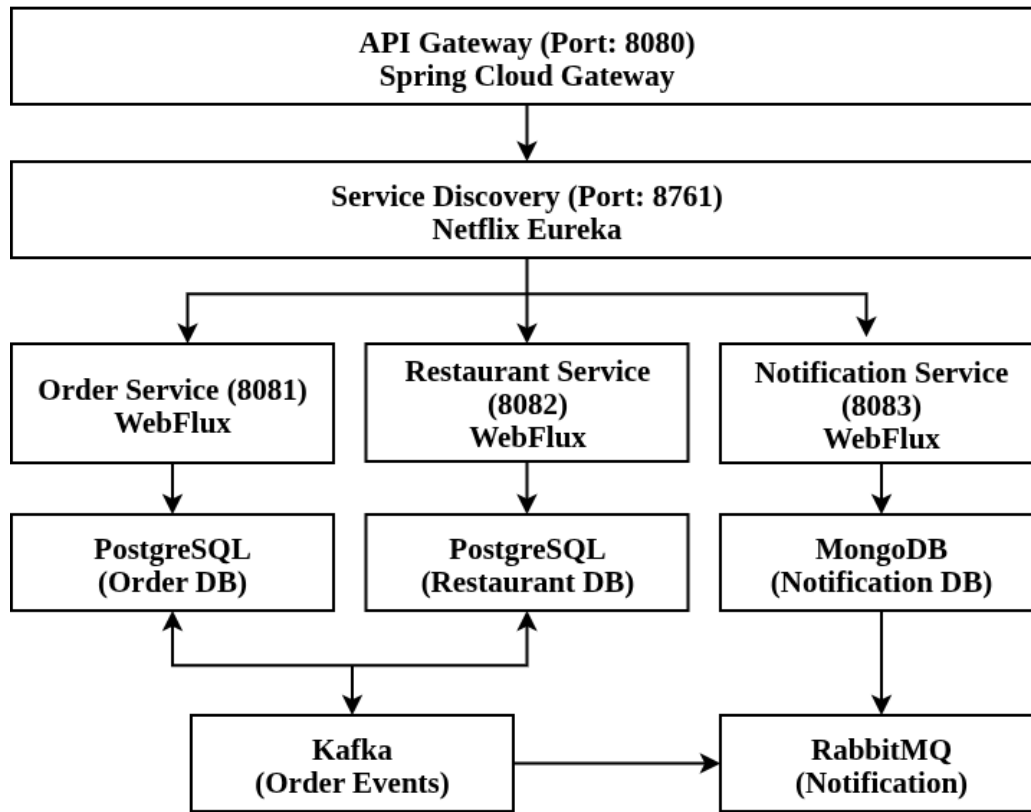


Figure 1: System Architecture Diagram

2.1 Microservices Architecture

The system follows a microservices architecture with the following components:

Core Microservices (≥ 3 services):

- **Order Service (Port 8081):** Manages orders, order status, and order processing. Built with Spring WebFlux and PostgreSQL.
- **Restaurant Service (Port 8082):** Manages restaurants, menu items, and restaurant information. Built with Spring WebFlux and PostgreSQL.
- **Notification Service (Port 8083):** Manages notifications, email/SMS alerts, and user communications. Built with Spring WebFlux and MongoDB.

Supporting Services:

- **Service Discovery (Eureka - Port 8761):** Handles service registration and discovery.
- **API Gateway (Port 8080):** Single entry point, request routing, and load balancing.

2.2 Communication Patterns

- The services communicate through:
- Synchronous REST APIs: For direct service-to-service communication
- Asynchronous Messaging: Using Apache Kafka for event-driven communication (Order → Kafka → Notification)
- Message Queuing: Using RabbitMQ for reliable message processing (Notification → RabbitMQ)

2.3 Data Flow

When a user creates an order:

1. Order Service receives the request
2. Order is saved in PostgreSQL database
3. Order Service publishes event to Kafka topic order-events
4. Notification Service consumes the Kafka event
5. Notification Service creates a notification record in MongoDB
6. Notification Service sends message to RabbitMQ queue
7. RabbitMQ consumer processes the notification

3. Implementation Details

3.1 Order Service

The Order Service is the core service responsible for managing food delivery orders. It provides comprehensive CRUD operations and order lifecycle management.

Key Features:

- Create orders with restaurant and menu item details
- Retrieve orders by ID or user
- Update order status (PENDING → CONFIRMED → PREPARING → OUT_FOR_DELIVERY → DELIVERED)
- Cancel pending orders
- Pagination support for order listing

Technologies Used:

- Spring WebFlux (Reactive)
- Spring Data R2DBC (Reactive PostgreSQL)
- Apache Kafka (Event publishing)
- MapStruct (DTO mapping)

Repository Features:

- Derived Queries: `findByUserId()`, `countByStatus()`
- Custom Queries: `@Query` with pagination and filtering
- Pagination: Pageable support

3.2 Restaurant Service

The Restaurant Service manages restaurant information and menu items. It provides comprehensive management of restaurant profiles and their associated menu items.

Key Features:

- Create and manage restaurants
- Add, update, and delete menu items
- Filter restaurants by cuisine type
- Search restaurants by name
- Rate and review restaurants
- Toggle restaurant availability

Technologies Used:

- Spring WebFlux (Reactive)
- Spring Data R2DBC (Reactive PostgreSQL)
- MapStruct (DTO mapping)

Repository Features:

- Derived Queries: `findByCuisineType()`, `findByIsActiveTrue()`
- Custom Queries: `searchRestaurantsByName()`, `findTopRestaurants()`
- Pagination: Pageable support

3.3 Notification Service

The Notification Service handles all user notifications, including email alerts, SMS notifications, and push notifications. It uses Kafka and RabbitMQ for asynchronous message processing.

Key Features:

- Create notifications for users
- Retrieve notifications by user or order
- Track notification status (PENDING → SENT → FAILED)
- Retry failed notifications
- Priority-based processing

Technologies Used:

- Spring WebFlux (Reactive)
- Spring Data MongoDB Reactive
- Apache Kafka (Consumer)
- RabbitMQ (Producer and Consumer)

Message Flow:

1. Order Service publishes event to Kafka (order-events topic)
2. Notification Service consumes Kafka event
3. Notification Service creates notification record in MongoDB
4. Notification Service sends message to RabbitMQ (notification.queue)
5. RabbitMQ consumer processes the notification (simulates email/SMS sending)
6. Notification status updates to SENT

3.4 DTO Mapping with MapStruct

MapStruct is used for efficient and type-safe DTO mapping. Each service has dedicated mappers:

- OrderMapper: Maps OrderRequest ↔ Order ↔ OrderResponse
- RestaurantMapper: Maps RestaurantRequest ↔ Restaurant ↔ RestaurantResponse
- MenuItemMapper: Maps MenuItemRequest ↔ MenuItem ↔ MenuItemResponse
- NotificationMapper: Maps NotificationRequest ↔ Notification ↔ NotificationResponse

MapStruct generates compile-time mapping code, reducing runtime overhead and ensuring type safety.

3.5 API Documentation

All APIs are documented using OpenAPI Specification with SpringDoc. Each service has Swagger UI accessible at:

- Order Service: <http://localhost:8081/swagger-ui.html>
- Restaurant Service: <http://localhost:8082/swagger-ui.html>
- Notification Service: <http://localhost:8083/swagger-ui.html>

4. Cloud-Native Concepts

4.1 Service Discovery (Netflix Eureka)

Service discovery is implemented using Netflix Eureka, enabling dynamic service registration and discovery [6]. All microservices register themselves with Eureka upon startup, and the API Gateway uses Eureka to locate services for request routing.

4.2 API Gateway (Spring Cloud Gateway)

The gateway provides request routing, load balancing, and cross-cutting concerns [7].

- /api/orders/** → Order Service
- /api/restaurants/** → Restaurant Service
- /api/notifications/** → Notification Service

4.3 API Security (JWT)

JWT-based security is implemented as a bonus feature [10]. Users authenticate via /api/auth/login and receive a JWT token, which must be included in subsequent API requests.

4.4 Centralized Configuration

Spring Cloud Config Server provides centralized configuration management for all microservices [11]. Configuration properties are stored in a Git repository and served to services at runtime.

4.5 Resilience (Resilience4j)

Resilience4j provides circuit breaker, retry, and rate limiter capabilities to handle failures gracefully [12].

4.6 Distributed Tracing (Zipkin)

Zipkin provides distributed tracing across microservices, allowing monitoring of request flows and performance analysis [13].

5. Message-Driven Communication

5.1 Kafka Integration

Kafka enables event-driven communication [14]:

1. Order Service publishes event to order-events topic
2. Notification Service consumes the event
3. Notification creates notification record

Benefits: decoupled services, asynchronous communication, event replay, high throughput.

5.2 RabbitMQ Integration

RabbitMQ provides reliable message processing [15]:

1. Notification Service sends message to notification.queue
2. Consumer processes the message
3. Status updates to SENT or FAILED

Benefits: message persistence, dead-letter handling, reliable delivery, work queue pattern.

6. Testing

6.1 Unit Testing

Unit tests are implemented using JUnit 5, Mockito, and StepVerifier for reactive streams [17].

Test Coverage:

- Service Layer: Business logic validation
- Controller Layer: API endpoint testing
- Repository Layer: Database operations testing

Service	Tests	Passed	Failed
Order Service	6	6	0
Restaurant Service	11	11	0
Notification Service	9	9	0
Total	26	26	0

Coverage: Service layer, Controller layer, Repository layer.

6.2 Integration Testing

Integration tests verify the complete flow across services:

- Order creation triggers Kafka event
- Notification Service consumes Kafka event
- Notification Service sends RabbitMQ message
- RabbitMQ consumer processes notification

6.3 Semi-Manual Testing

Test cases with expected and actual results are documented in docs/Test_Cases.md.

Test ID	API Endpoint	Input	Expected Output	Actual Output	Status
IT-001	POST /api/restaurants	Valid restaurant data	201 Created	201 Created	PASS
IT-002	POST /api/orders	Valid order data	201 Created	201 Created	PASS
IT-003	GET /api/orders/{id}	Valid order ID	200 OK	200 OK	PASS
IT-004	Kafka Event	Order created	Event published to order-events	Event received	PASS
IT-005	RabbitMQ	Notification sent	Message in notification.queue	Message processed	PASS

7. Docker Containerization

7.1 Container Architecture

All services are containerized using Docker. The following containers are running:

ahtuh-in@tuh-in:~/Documents/Spring 2026/Software Service Engineering/Final/Food-delivery-system\$ docker ps						
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	NAMES	PORTS
6b3611e13c87	openzipkin/zipkin:latest	"start-zipkin"	12 minutes ago	Up 12 minutes (healthy)	zipkin	9410/tcp, 0.0.0.0:9411->9411/tcp, [::]:9411->9411/tcp
5bdaac551dc3	rabbitmq:3-management	"docker-entrypoint.s..."	13 minutes ago	Up 12 minutes	rabbitmq	4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
9d6374db6a97	confluentinc/cp-kafka:7.0.0	"/etc/confluent/dock..."	13 minutes ago	Up 13 minutes	kafka	0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
b6705e3951f0	zookeeper:3.8	"/docker-entrypoint.s..."	13 minutes ago	Up 13 minutes	zookeeper	2888/tcp, 3888/tcp, 0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp, 8080/tcp
2181->2181/tcp, f8c5f7dbeca4	mongo:7	"docker-entrypoint.s..."	13 minutes ago	Up 13 minutes	mongodb	0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
262c4ea71b6d	postgres:16	"docker-entrypoint.s..."	13 minutes ago	Up 13 minutes	postgres-restaurant	0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
08db05a72370	postgres:16	"docker-entrypoint.s..."	13 minutes ago	Up 13 minutes	postgres-order	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp

Figure 2: Docker Containers Running

Container	Port	Purpose
postgres-order	5432	Order Database
postgres-restaurant	5433	Restaurant Database
mongodb	27017	Notification Database

zookeeper	2181	Kafka Coordination
kafka	9092	Message Broker
rabbitmq	5672/15672	Message Queue
zipkin	9411	Tracing
service-discovery	8761	Eureka
api-gateway	8080	Gateway
order-service	8081	Order Service
restaurant-service	8082	Restaurant Service
notification-service	8083	Notification Service

7.2 Docker Compose

Docker Compose orchestrates all services with dependency management, environment variables, volume mounting, and network isolation.

8. Challenges and Solutions

Challenge 1: Learning and implementing reactive programming with Spring WebFlux.

Solution: Followed Spring WebFlux documentation, used StepVerifier for testing, and implemented non-blocking database access with R2DBC.

Challenge 2: Configuring Kafka and RabbitMQ for reliable message delivery.

Solution: Used Docker containers for local development, implemented retry mechanisms, and added error handling for failed messages.

Challenge 3: Managing database schemas across multiple services.

Solution: Used schema.sql files for initialization, implemented CREATE TABLE IF NOT EXISTS statements, and followed database-per-service pattern.

Challenge 4: Implementing JWT authentication with Spring Security WebFlux.

Solution: Created custom authentication filters, implemented token generation and validation, and configured security for reactive endpoints.

9. Conclusion

The Food Delivery Management System successfully demonstrates the implementation of a cloud-native reactive microservices architecture using Spring Boot and Spring Cloud technologies.

Key Achievements:

- 3 independent reactive microservices
- Service discovery with Netflix Eureka
- API Gateway for request routing
- Database per service pattern (PostgreSQL + MongoDB)
- Kafka and RabbitMQ integration
- JWT security
- Resilience with Resilience4j
- Distributed tracing with Zipkin
- Docker containerization
- 26 unit tests passing

Future Improvements:

- Implement OAuth2 authentication
- Add real-time order tracking with WebSocket
- Implement payment gateway integration
- Add comprehensive monitoring and alerting
- Implement CI/CD pipeline with GitHub Actions

This project provides a solid foundation for building scalable, maintainable, and resilient microservices applications in production environments.

10. References

[1] Spring Boot Documentation. "Spring Boot Reference Guide." Spring.io. [Online]. Available: <https://spring.io/projects/spring-boot>

[2] Spring Cloud Documentation. "Spring Cloud Reference Guide." Spring.io. [Online]. Available: <https://spring.io/projects/spring-cloud>

[3] Spring WebFlux Documentation. "Reactive Programming with Spring WebFlux." Spring.io. [Online]. Available: <https://docs.spring.io/spring-framework/reference/web/webflux.html>

[4] Newman, S. (2015). "Building Microservices: Designing Fine-Grained Systems." O'Reilly Media.

- [5] Evans, E. (2003). "Domain-Driven Design: Tackling Complexity in the Heart of Software." Addison-Wesley.
- [6] Netflix Eureka. "Eureka Service Discovery." GitHub. [Online]. Available: <https://github.com/Netflix/eureka>
- [7] Spring Cloud Gateway. "API Gateway with Spring Cloud Gateway." Spring.io. [Online]. Available: <https://spring.io/projects/spring-cloud-gateway>
- [8] Richardson, C. (2018). "Microservices Patterns." Manning Publications.
- [9] MapStruct Documentation. "Java Bean Mappings." MapStruct.org. [Online]. Available: <https://mapstruct.org/>
- [10] Spring Security Documentation. "JWT Authentication with Spring Security." Spring.io. [Online]. Available: <https://spring.io/projects/spring-security>
- [11] Spring Cloud Config. "Centralized Configuration Management." Spring.io. [Online]. Available: <https://spring.io/projects/spring-cloud-config>
- [12] Resilience4j Documentation. "Circuit Breaker and Resilience." Resilience4j.io. [Online]. Available: <https://resilience4j.readme.io/docs>
- [13] Zipkin Documentation. "Distributed Tracing with Zipkin." Zipkin.io. [Online]. Available: <https://zipkin.io/pages/documentation.html>
- [14] Apache Kafka Documentation. "Event Streaming Platform." Apache.org. [Online]. Available: <https://kafka.apache.org/documentation/>
- [15] RabbitMQ Documentation. "Message Queueing System." RabbitMQ.com. [Online]. Available: <https://www.rabbitmq.com/documentation.html>
- [16] JUnit 5 Documentation. "Testing Framework." JUnit.org. [Online]. Available: <https://junit.org/junit5/docs/current/user-guide/>
- [17] Docker Documentation. "Containerization Platform." Docker.com. [Online]. Available: <https://docs.docker.com/>